

Компьютерная графика

Практика 5: Bind groups, uniform буферы, текстуры, сэмплеры

2026

- Начиная с этой практики можно всю пользоваться математическими библиотеками (`glam` в Rust, `math` в C++)
- **NB:** в Rust `glam` матрицы хранятся по столбцам (как и ожидает WebGPU), в C++ `math` – по строкам (надо их транспонировать, либо умножать в шейдере с другой стороны)



Задание 1

Создаём uniform buffer и bind группу

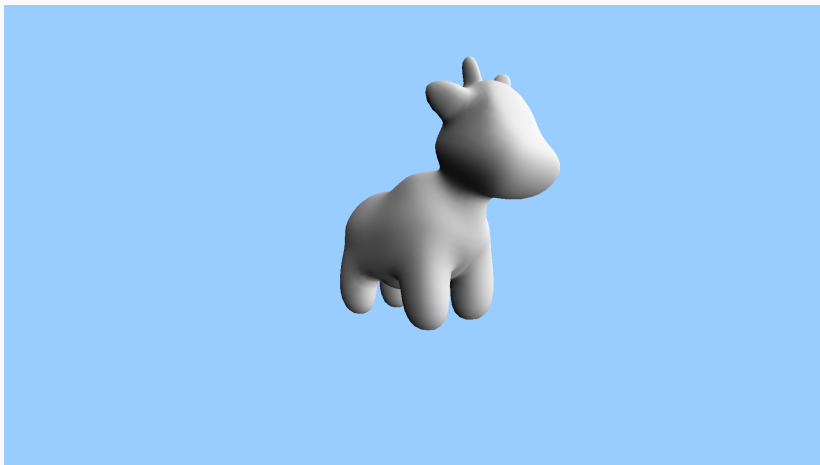
- Размер буфера – как у immediates, `usage = Uniform`
- Создаём bind group layout с одним entry: `binding = 0`, `visibility = Vertex`, `buffer.type = Uniform`
- Создаём bind group с этим layout, тоже с одним entry: `binding = 0`, `buffer` = наш буфер
- **NB:** Пока ничего не изменится, мы никак не используем этот буфер :)

Задание 2

Используем uniform buffer

- Убираем `immediateSize` из pipeline layout'a
- Добавляем наш bind group layout в pipeline layout
- Запись immediates в render pass'e заменяем на запись в наш буфер (`queue.write_buffer`) **перед созданием** render pass'a (в том же формате: сначала матрица `model`, затем `viewProjection`)
- Добавляем `render_pass.set_bind_group(0, bind_group)` перед вызовом `draw_indexed()`
- В шейдере заменяем `var<immediate>` на `@group(0) @binding(0) var<uniform>`
- Если всё сделать правильно, картинка не изменится – мы берём те же матрицы, но теперь из другого места

Задание 2



Добавляем текстурные координаты

- Добавляем в render pipeline ещё один атрибут вершины, `shaderLocation = 2`, формат `vec2f` (в массиве вершин он уже есть)
- Передаём его из вершинного шейдера во фрагментный
- Выводим его в качестве цвета, например как `vec4f(texcoord, 0.0, 1.0)`

Задание 3



Задание 4

Загружаем текстуру

- Создаём текстуру: `device.create_texture()`, `dimension = 2D`, `format = RGBA8`, `usage = CopyDst | TextureBinding`, размер ВЗЯТЬ ИЗ `cow_image`
- Загружаем пиксели в нашу текстуру: `queue.write_texture()`, `aspect = All`, `bytes_per_row = cow_image.width * 4`, `rows_per_image = cow_image.height`
- Создаём texture view (дескриптор можно оставить пустым – все дефолтные настройки нам подойдут)
- **NB:** Картинка не изменится, мы никак не используем текстуру :)

Задание 5

Создаём сэмплер и используем текстуру

- Добавляем ещё два entry в bind group layout:
 - `binding = 1, visibility = Fragment,`
`texture.view_dimension = 2D, texture.sample_type = Float`
 - `binding = 2, visibility = Fragment,`
`sampler.type = Filtering`
- Создаём sampler: `device.create_sampler()`,
`address_mode = Repeat, mag_filter = Nearest,`
`min_filter = linear, mipmap_filter = Nearest`
- Добавляем texture view и sampler в bind группу
- В шейдере:
 - Возвращаем нормальный цвет вместо текстурных координат
 - Добавляем текстуру и сэмплер
 - Записываем результат вызова `textureSample(...)` в переменную `albedo`

Задание 5



Задание 6

Загружаем mipmap

- При создании текстуры указываем `mip_level_count = 4`
- Нулевой уровень – сама основная текстура, она уже загружена
- Заполним оставшиеся 3 уровня красным, зелёным, и синим цветами, соответственно
- Каждый следующий уровень должен быть в 2 раза меньше предыдущего по обеим осям!
- Подвигайте камеру, чтобы увидеть, как включаются разные mipmap-уровни

Задание 6



Анимируем текстуру

- Добавим что-нибудь к текстурным координатам во фрагментном шейдере, чтобы текстура двигалась со временем
- Нужно будет как-то передать время в шейдер: можно как часть `uniform-buffer`'а, можно как `immediate`

Задание 7*

